MARAN PMS — Prompt 56: Bitácora de Incidencias del Sistema

Objetivo

Crear un registro simple y estructurado de problemas detectados durante la operación del sistema. Permite que cualquier usuario reporte un error, que el equipo técnico lo siga, y que quede trazabilidad de la resolución.

Archivos a modificar/crear

- `shared/schema.ts` **— nueva tabla** `system_incidents`
- `server/routes.ts` **— 5 endpoints nuevos bajo** `/api/incidents`
- `client/src/pages/administration.tsx` **— nuevo tab "Bitácora"**

PARTE 1 — Schema (shared/schema.ts)

Agregar al final del archivo, antes del último export:

```
// ==================== SYSTEM INCIDENTS ====================

export type IncidentSeverity = "baja" | "media" | "alta" | "critica";
export type IncidentStatus = "pendiente" | "en_revision" | "resuelto" | "descarta
export type IncidentModule =
  | "planning" | "reservas" | "check-in" | "check-out"
  | "grupos" | "restaurant" | "spa" | "eventos"
  | "housekeeping" | "hospitalidad" | "inventario"
  | "cajas" | "reportes" | "administracion" | "otro";

export const systemIncidents = pgTable("system_incidents", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),

  // Datos del incidente
  title: text("title").notNull(),
  description: text("description").notNull(),
  module: text("module").$type<IncidentModule>().notNull().default("otro"),
```

```
  severity: text("severity").$type<IncidentSeverity>().notNull().default("media")
  status: text("status").$type<IncidentStatus>().notNull().default("pendiente"),

  // Quién reportó
  reportedBy: text("reported_by").notNull(),
  reportedAt: timestamp("reported_at").notNull().defaultNow(),

  // Seguimiento
  assignedTo: text("assigned_to"),
  resolvedBy: text("resolved_by"),
  resolvedAt: timestamp("resolved_at"),
  resolutionNotes: text("resolution_notes"),

  // Captura / adjunto (URL o descripción)
  screenshotUrl: text("screenshot_url"),

  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const insertSystemIncidentSchema = createInsertSchema(systemIncidents).omi
  id: true,
  createdAt: true,
  updatedAt: true,
  resolvedAt: true,
});

export type InsertSystemIncident = z.infer<typeof insertSystemIncidentSchema>;
export type SystemIncident = typeof systemIncidents.$inferSelect;
```

PARTE 2 — Backend (server/routes.ts)

Agregar dentro de `registerRoutes`, junto a los otros endpoints de admin:

```
// ==================== SYSTEM INCIDENTS ====================

// Listar incidentes (con filtros opcionales)
app.get("/api/incidents", requireAuth, async (req, res) => {
  try {
    const { status, severity, module } = req.query;

    let query = db.select().from(systemIncidents);

    const conditions = [];
```

```
    if (status && status !== "all") conditions.push(eq(systemIncidents.status, st
    if (severity && severity !== "all") conditions.push(eq(systemIncidents.severi
    if (module && module !== "all") conditions.push(eq(systemIncidents.module, mo

    const incidents = await db
      .select()
      .from(systemIncidents)
      .where(conditions.length > 0 ? and(...conditions) : undefined)
      .orderBy(desc(systemIncidents.reportedAt));

    res.json(incidents);
  } catch (error) {
    res.status(500).json({ error: "Error fetching incidents" });
  }
});

// Crear nuevo incidente
app.post("/api/incidents", requireAuth, async (req, res) => {
  try {
    const { title, description, module, severity, reportedBy, screenshotUrl } = r

    if (!title || !description || !reportedBy) {
      return res.status(400).json({ error: "Título, descripción y quien reporta s
    }

    const [incident] = await db.insert(systemIncidents).values({
      id: randomUUID(),
      title,
      description,
      module: module || "otro",
      severity: severity || "media",
      status: "pendiente",
      reportedBy,
      reportedAt: new Date(),
      screenshotUrl: screenshotUrl || null,
    }).returning();

    res.status(201).json(incident);
  } catch (error) {
    res.status(500).json({ error: "Error creating incident" });
  }
});

// Actualizar estado de un incidente
app.patch("/api/incidents/:id", requireAuth, async (req, res) => {
  try {
    const { status, assignedTo, resolvedBy, resolutionNotes } = req.body;
```

```
    const updateData: any = { updatedAt: new Date() };
    if (status) updateData.status = status;
    if (assignedTo !== undefined) updateData.assignedTo = assignedTo;
    if (resolvedBy) updateData.resolvedBy = resolvedBy;
    if (resolutionNotes !== undefined) updateData.resolutionNotes = resolutionNot

    // Si se marca como resuelto, registrar fecha
    if (status === "resuelto" || status === "descartado") {
      updateData.resolvedAt = new Date();
    }

    const [updated] = await db
      .update(systemIncidents)
      .set(updateData)
      .where(eq(systemIncidents.id, req.params.id))
      .returning();

    if (!updated) return res.status(404).json({ error: "Incidente no encontrado"

    res.json(updated);
  } catch (error) {
    res.status(500).json({ error: "Error updating incident" });
  }
});

// Eliminar incidente (solo admin)
app.delete("/api/incidents/:id", requireAuth, async (req, res) => {
  try {
    await db.delete(systemIncidents).where(eq(systemIncidents.id, req.params.id))
    res.json({ success: true });
  } catch (error) {
    res.status(500).json({ error: "Error deleting incident" });
  }
});

// Resumen de incidentes (para el tablero)
app.get("/api/incidents/summary", requireAuth, async (req, res) => {
  try {
    const all = await db.select().from(systemIncidents);

    const summary = {
      total: all.length,
      pendiente: all.filter(i => i.status === "pendiente").length,
      en_revision: all.filter(i => i.status === "en_revision").length,
      resuelto: all.filter(i => i.status === "resuelto").length,
      criticos: all.filter(i => i.severity === "critica" && i.status !== "resuelt
```

```
      altos: all.filter(i => i.severity === "alta" && i.status !== "resuelto" &&
    };

    res.json(summary);
  } catch (error) {
    res.status(500).json({ error: "Error fetching incident summary" });
  }
});
```

Importante: agregar `systemIncidents` a los imports de schema en routes.ts:

```
import { ..., systemIncidents } from "@shared/schema";
```

PARTE 3 — Frontend (client/src/pages/administration.tsx)

3.1 Agregar el tab en TabsList

```
// Después del tab "audit":
<TabsTrigger value="incidencias" data-testid="tab-admin-incidencias">
  <AlertTriangle className="h-4 w-4 mr-1" />
  Bitácora
</TabsTrigger>
```

3.2 TabsContent completo de Bitácora

```
<TabsContent value="incidencias" className="space-y-4">
  <IncidenciasTab />
</TabsContent>
```

3.3 Componente IncidenciasTab

Agregar antes del `export default function AdministrationPage()` **:**

```
function IncidenciasTab() {
  const { toast } = useToast();
  const [filterStatus, setFilterStatus] = useState("all");
  const [filterSeverity, setFilterSeverity] = useState("all");
  const [filterModule, setFilterModule] = useState("all");
  const [isNewDialogOpen, setIsNewDialogOpen] = useState(false);
  const [isDetailDialogOpen, setIsDetailDialogOpen] = useState(false);
```

```
const [selectedIncident, setSelectedIncident] = useState<any>(null);

// Formulario nuevo incidente
const [newTitle, setNewTitle] = useState("");
const [newDesc, setNewDesc] = useState("");
const [newModule, setNewModule] = useState("otro");
const [newSeverity, setNewSeverity] = useState("media");
const [newReportedBy, setNewReportedBy] = useState("");

// Formulario actualización
const [updateStatus, setUpdateStatus] = useState("");
const [updateAssigned, setUpdateAssigned] = useState("");
const [updateResolution, setUpdateResolution] = useState("");
const [updateResolvedBy, setUpdateResolvedBy] = useState("");

const queryParams = new URLSearchParams();
if (filterStatus !== "all") queryParams.set("status", filterStatus);
if (filterSeverity !== "all") queryParams.set("severity", filterSeverity);
if (filterModule !== "all") queryParams.set("module", filterModule);

const { data: incidents = [], isLoading, refetch } = useQuery<any[]>({
  queryKey: ["/api/incidents", filterStatus, filterSeverity, filterModule],
  queryFn: async () => {
    const res = await fetch(`/api/incidents?${queryParams.toString()}`, { crede
    if (!res.ok) throw new Error("Error cargando incidentes");
    return res.json();
  },
});

const { data: summary } = useQuery<any>({
  queryKey: ["/api/incidents/summary"],
  queryFn: async () => {
    const res = await fetch("/api/incidents/summary", { credentials: "include"
    if (!res.ok) return {};
    return res.json();
  },
});

const createMutation = useMutation({
  mutationFn: async (data: any) => {
    const res = await fetch("/api/incidents", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "include",
      body: JSON.stringify(data),
    });
    if (!res.ok) throw new Error("Error al crear incidente");
```

```
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/incidents"] });
    queryClient.invalidateQueries({ queryKey: ["/api/incidents/summary"] });
    setIsNewDialogOpen(false);
    setNewTitle(""); setNewDesc(""); setNewModule("otro");
    setNewSeverity("media"); setNewReportedBy("");
    toast({ title: "Incidente registrado" });
  },
  onError: () => toast({ title: "Error", description: "No se pudo registrar el
});

const updateMutation = useMutation({
  mutationFn: async ({ id, data }: { id: string; data: any }) => {
    const res = await fetch(`/api/incidents/${id}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      credentials: "include",
      body: JSON.stringify(data),
    });
    if (!res.ok) throw new Error("Error al actualizar");
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/incidents"] });
    queryClient.invalidateQueries({ queryKey: ["/api/incidents/summary"] });
    setIsDetailDialogOpen(false);
    toast({ title: "Incidente actualizado" });
  },
});

const SEVERITY_CONFIG: Record<string, { label: string; color: string }> = {
  baja:    { label: "Baja",    color: "bg-green-100 text-green-700 dark:bg-gree
  media:   { label: "Media",   color: "bg-yellow-100 text-yellow-700 dark:bg-ye
  alta:    { label: "Alta",    color: "bg-orange-100 text-orange-700 dark:bg-or
  critica: { label: "Crítica", color: "bg-red-100 text-red-700 dark:bg-red-900/
};

const STATUS_CONFIG: Record<string, { label: string; color: string }> = {
  pendiente:   { label: "Pendiente",   color: "bg-gray-100 text-gray-700 dark:b
  en_revision: { label: "En revisión", color: "bg-blue-100 text-blue-700 dark:b
  resuelto:    { label: "Resuelto",    color: "bg-green-100 text-green-700 dark
  descartado:  { label: "Descartado",  color: "bg-gray-100 text-gray-500 dark:b
};

const MODULES = [
```

```
  "planning", "reservas", "check-in", "check-out", "grupos",
  "restaurant", "spa", "eventos", "housekeeping", "hospitalidad",
  "inventario", "cajas", "reportes", "administracion", "otro"
];

const openDetail = (incident: any) => {
  setSelectedIncident(incident);
  setUpdateStatus(incident.status);
  setUpdateAssigned(incident.assignedTo || "");
  setUpdateResolution(incident.resolutionNotes || "");
  setUpdateResolvedBy(incident.resolvedBy || "");
  setIsDetailDialogOpen(true);
};

return (
  <div className="space-y-6">

    {/* Resumen */}
    <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
      <Card className="border-l-4 border-l-gray-400">
        <CardContent className="p-4">
          <p className="text-2xl font-bold">{summary?.pendiente ?? 0}</p>
          <p className="text-xs text-muted-foreground">Pendientes</p>
        </CardContent>
      </Card>
      <Card className="border-l-4 border-l-blue-400">
        <CardContent className="p-4">
          <p className="text-2xl font-bold">{summary?.en_revision ?? 0}</p>
          <p className="text-xs text-muted-foreground">En revisión</p>
        </CardContent>
      </Card>
      <Card className="border-l-4 border-l-red-400">
        <CardContent className="p-4">
          <p className="text-2xl font-bold">{summary?.criticos ?? 0}</p>
          <p className="text-xs text-muted-foreground">Críticos abiertos</p>
        </CardContent>
      </Card>
      <Card className="border-l-4 border-l-green-400">
        <CardContent className="p-4">
          <p className="text-2xl font-bold">{summary?.resuelto ?? 0}</p>
          <p className="text-xs text-muted-foreground">Resueltos</p>
        </CardContent>
      </Card>
    </div>

    {/* Filtros + botón nuevo */}
    <div className="flex flex-wrap items-center gap-3">
```

```
  <Select value={filterStatus} onValueChange={setFilterStatus}>
    <SelectTrigger className="w-36"><SelectValue placeholder="Estado" /></S
    <SelectContent>
      <SelectItem value="all">Todos</SelectItem>
      <SelectItem value="pendiente">Pendiente</SelectItem>
      <SelectItem value="en_revision">En revisión</SelectItem>
      <SelectItem value="resuelto">Resuelto</SelectItem>
      <SelectItem value="descartado">Descartado</SelectItem>
    </SelectContent>
  </Select>

  <Select value={filterSeverity} onValueChange={setFilterSeverity}>
    <SelectTrigger className="w-36"><SelectValue placeholder="Gravedad" /><
    <SelectContent>
      <SelectItem value="all">Todas</SelectItem>
      <SelectItem value="critica">Crítica</SelectItem>
      <SelectItem value="alta">Alta</SelectItem>
      <SelectItem value="media">Media</SelectItem>
      <SelectItem value="baja">Baja</SelectItem>
    </SelectContent>
  </Select>

  <Select value={filterModule} onValueChange={setFilterModule}>
    <SelectTrigger className="w-40"><SelectValue placeholder="Módulo" /></S
    <SelectContent>
      <SelectItem value="all">Todos los módulos</SelectItem>
      {MODULES.map(m => (
        <SelectItem key={m} value={m}>{m.charAt(0).toUpperCase() + m.slice(
      ))}
    </SelectContent>
  </Select>

  <div className="ml-auto">
    <Button onClick={() => setIsNewDialogOpen(true)} data-testid="btn-new-i
      <Plus className="h-4 w-4 mr-2" />
      Reportar incidencia
    </Button>
  </div>
</div>

{/* Lista de incidentes */}
<Card>
  <CardContent className="p-0">
    {isLoading ? (
      <div className="p-8 text-center text-muted-foreground">Cargando...</d
    ) : incidents.length === 0 ? (
      <div className="p-8 text-center text-muted-foreground">
```

```
    No hay incidencias registradas con los filtros actuales
  </div>
) : (
  <Table>
    <TableHeader>
      <TableRow>
        <TableHead>Título</TableHead>
        <TableHead>Módulo</TableHead>
        <TableHead>Gravedad</TableHead>
        <TableHead>Estado</TableHead>
        <TableHead>Reportado por</TableHead>
        <TableHead>Fecha</TableHead>
        <TableHead className="w-10"></TableHead>
      </TableRow>
    </TableHeader>
    <TableBody>
      {incidents.map((incident: any) => (
        <TableRow
          key={incident.id}
          className="cursor-pointer hover:bg-muted/50"
          onClick={() => openDetail(incident)}
          data-testid={`incident-row-${incident.id}`}
        >
          <TableCell className="font-medium max-w-[200px] truncate">
            {incident.title}
          </TableCell>
          <TableCell className="capitalize text-sm">{incident.module}</
          <TableCell>
            <Badge className={`text-xs ${SEVERITY_CONFIG[incident.sever
              {SEVERITY_CONFIG[incident.severity]?.label ?? incident.se
            </Badge>
          </TableCell>
          <TableCell>
            <Badge className={`text-xs ${STATUS_CONFIG[incident.status]
              {STATUS_CONFIG[incident.status]?.label ?? incident.status
            </Badge>
          </TableCell>
          <TableCell className="text-sm">{incident.reportedBy}</TableCe
          <TableCell className="text-sm text-muted-foreground">
            {new Date(incident.reportedAt).toLocaleDateString("es-AR")}
          </TableCell>
          <TableCell>
            <ChevronRight className="h-4 w-4 text-muted-foreground" />
          </TableCell>
        </TableRow>
      ))}
    </TableBody>
```

```
      </Table>
    )}
  </CardContent>
</Card>

{/* Dialog — Nuevo incidente */}
<Dialog open={isNewDialogOpen} onOpenChange={setIsNewDialogOpen}>
  <DialogContent className="max-w-lg">
    <DialogHeader>
      <DialogTitle>Reportar incidencia</DialogTitle>
    </DialogHeader>
    <div className="space-y-4">
      <div>
        <Label>Título *</Label>
        <Input
          value={newTitle}
          onChange={e => setNewTitle(e.target.value)}
          placeholder="Descripción breve del problema"
          data-testid="input-incident-title"
        />
      </div>
      <div>
        <Label>Descripción detallada *</Label>
        <Textarea
          value={newDesc}
          onChange={e => setNewDesc(e.target.value)}
          placeholder="¿Qué pasó? ¿Cómo reproducirlo? ¿Qué se esperaba?"
          rows={4}
          data-testid="input-incident-desc"
        />
      </div>
      <div className="grid grid-cols-2 gap-3">
        <div>
          <Label>Módulo</Label>
          <Select value={newModule} onValueChange={setNewModule}>
            <SelectTrigger data-testid="select-incident-module">
              <SelectValue />
            </SelectTrigger>
            <SelectContent>
              {MODULES.map(m => (
                <SelectItem key={m} value={m}>{m.charAt(0).toUpperCase() +
              ))}
            </SelectContent>
          </Select>
        </div>
        <div>
          <Label>Gravedad</Label>
```

```
            <Select value={newSeverity} onValueChange={setNewSeverity}>
              <SelectTrigger data-testid="select-incident-severity">
                <SelectValue />
              </SelectTrigger>
              <SelectContent>
                <SelectItem value="baja">Baja — no bloquea operación</SelectI
                <SelectItem value="media">Media — dificulta operación</Select
                <SelectItem value="alta">Alta — bloquea parte del sistema</Se
                <SelectItem value="critica">Crítica — sistema caído</SelectIt
              </SelectContent>
            </Select>
          </div>
        </div>
        <div>
          <Label>Reportado por *</Label>
          <Input
            value={newReportedBy}
            onChange={e => setNewReportedBy(e.target.value)}
            placeholder="Tu nombre"
            data-testid="input-incident-reporter"
          />
        </div>
      </div>
      <DialogFooter>
        <Button variant="outline" onClick={() => setIsNewDialogOpen(false)}>C
        <Button
          onClick={() => createMutation.mutate({
            title: newTitle, description: newDesc,
            module: newModule, severity: newSeverity, reportedBy: newReported
          })}
          disabled={!newTitle.trim() || !newDesc.trim() || !newReportedBy.tri
          data-testid="btn-submit-incident"
        >
          {createMutation.isPending ? "Guardando..." : "Reportar"}
        </Button>
      </DialogFooter>
    </DialogContent>
  </Dialog>

  {/* Dialog — Detalle y actualización */}
  <Dialog open={isDetailDialogOpen} onOpenChange={setIsDetailDialogOpen}>
    <DialogContent className="max-w-lg">
      <DialogHeader>
        <DialogTitle className="flex items-center gap-2">
          <AlertTriangle className="h-5 w-5" />
          {selectedIncident?.title}
        </DialogTitle>
```

```
</DialogHeader>
{selectedIncident && (
  <div className="space-y-4">
    <div className="flex gap-2">
      <Badge className={SEVERITY_CONFIG[selectedIncident.severity]?.col
        {SEVERITY_CONFIG[selectedIncident.severity]?.label}
      </Badge>
      <Badge variant="outline" className="capitalize">{selectedIncident
      <Badge className={STATUS_CONFIG[selectedIncident.status]?.color}>
        {STATUS_CONFIG[selectedIncident.status]?.label}
      </Badge>
    </div>

    <div className="text-sm text-muted-foreground border rounded-lg p-3
      {selectedIncident.description}
    </div>

    <div className="text-xs text-muted-foreground">
      Reportado por <strong>{selectedIncident.reportedBy}</strong> el{"
      {new Date(selectedIncident.reportedAt).toLocaleString("es-AR")}
    </div>

    <div className="border-t pt-4 space-y-3">
      <p className="text-sm font-medium">Actualizar estado</p>
      <div className="grid grid-cols-2 gap-3">
        <div>
          <Label className="text-xs">Estado</Label>
          <Select value={updateStatus} onValueChange={setUpdateStatus}>
            <SelectTrigger><SelectValue /></SelectTrigger>
            <SelectContent>
              <SelectItem value="pendiente">Pendiente</SelectItem>
              <SelectItem value="en_revision">En revisión</SelectItem>
              <SelectItem value="resuelto">Resuelto</SelectItem>
              <SelectItem value="descartado">Descartado</SelectItem>
            </SelectContent>
          </Select>
        </div>
        <div>
          <Label className="text-xs">Asignado a</Label>
          <Input
            value={updateAssigned}
            onChange={e => setUpdateAssigned(e.target.value)}
            placeholder="Responsable"
          />
        </div>
      </div>
      {(updateStatus === "resuelto" || updateStatus === "descartado") &
```

```
                  <div>
                    <Label className="text-xs">Resuelto por</Label>
                    <Input
                      value={updateResolvedBy}
                      onChange={e => setUpdateResolvedBy(e.target.value)}
                      placeholder="Quien resolvió"
                    />
                  </div>
                )}
                <div>
                  <Label className="text-xs">Notas de resolución</Label>
                  <Textarea
                    value={updateResolution}
                    onChange={e => setUpdateResolution(e.target.value)}
                    placeholder="¿Cómo se resolvió? ¿Qué se cambió?"
                    rows={3}
                  />
                </div>
              </div>
            </div>
          )}
          <DialogFooter>
            <Button variant="outline" onClick={() => setIsDetailDialogOpen(false)
            <Button
              onClick={() => updateMutation.mutate({
                id: selectedIncident.id,
                data: {
                  status: updateStatus,
                  assignedTo: updateAssigned || null,
                  resolvedBy: updateResolvedBy || null,
                  resolutionNotes: updateResolution || null,
                },
              })}
              disabled={updateMutation.isPending}
            >
              {updateMutation.isPending ? "Guardando..." : "Guardar cambios"}
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>

    </div>
  );
}
```

3.4 Imports adicionales necesarios

Verificar que estén importados en `administration.tsx` **:**

```
import { AlertTriangle, ChevronRight, Plus } from "lucide-react";
import { Badge } from "@/components/ui/badge";
// Table, TableBody, TableCell, TableHead, TableHeader, TableRow — verificar
// Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter — verificar
// Select, SelectContent, SelectItem, SelectTrigger, SelectValue — verificar
// Label, Input, Textarea — verificar
```

NOTAS TÉCNICAS

- **Migración DB:** `drizzle-kit push` **para crear la tabla** `system_incidents`
- **No hay lógica compleja — es un CRUD puro con filtros**
- **El campo** `screenshotUrl` **está preparado para una futura integración de adjuntos, por ahora no se usa en la UI**
- **Los filtros se aplican en el backend — no cargar todos los incidentes en el frontend**
- **La tabla** `system_incidents` **no tiene foreign keys — es independiente de todos los módulos, lo que la hace simple y robusta**